

针对上面列出的相关问题，直接逐一给出完整的答案，并以文档形式输出

大厂前端面试核心考点完整答案文档

目录

- 1. JavaScript 核心原理
- 2. React 原理与生态
- 3. 算法与手写代码
- 4. 工程化 & 网络 & 性能

1. JavaScript 核心原理

1.1 事件循环

题目：说出下面代码的输出顺序，并解释原因

```
javascript

console.log(1);
setTimeout(() => console.log(2), 0);
Promise.resolve().then(() => console.log(3));
console.log(4);
```

答案：

```
text

输出顺序：1, 4, 3, 2
```

原理解释：

JavaScript 是单线程语言，事件循环是其处理异步的核心机制。

执行流程：

- 1. 执行同步代码： console.log(1) 和 console.log(4) 立即执行
- 2. 遇到宏任务： setTimeout 的回调放入宏任务队列
- 3. 遇到微任务： Promise.then 的回调放入微任务队列
- 4. 同步代码执行完毕，检查微任务队列
- 5. 执行所有微任务：输出 3
- 6. 执行一个宏任务：输出 2
- 7. 重复 4-6 步骤

微任务与宏任务对比：

类型	常见API	执行时机
微任务	Promise.then/catch/finally 、 MutationObserver 、 queueMicrotask	当前宏任务执行完后、下一个宏任务前
宏任务	setTimeout/setInterval 、 I/O 、 UI渲染 、 MessageChannel	每次事件循环取一个执行

进阶追问：Node.js 事件循环与浏览器有什么区别？

Node.js 的事件循环有六个阶段：

- timers: 执行 `setTimeout` / `setInterval`
- pending callbacks: 执行系统操作的回调
- idle, prepare: 内部使用
- poll: 检索新的 I/O 事件
- check: 执行 `setImmediate`
- close callbacks: 关闭回调

Node.js 中 `process.nextTick` 优先级高于微任务队列。

1.2 闭包

题目：什么是闭包？有什么应用场景？可能引起什么问题？

答案：

定义：闭包是指一个函数能够访问其外部函数作用域中的变量，即使外部函数已经执行完毕。

```
javascript

function outer() {
  let count = 0;
  return function inner() {
    count++;
    console.log(count);
  };
}

const counter = outer();
counter(); // 1
counter(); // 2
// inner 函数就是闭包，它持有对 count 的引用
```

应用场景：

1. 模块化模式：实现私有变量

```
javascript

const module = (function() {
  let privateVar = 0;
  return {
    increment: () => ++privateVar,
    get: () => privateVar
  };
})();
```

2. 函数工厂：动态生成函数

```
javascript

function multiplyBy(x) {
  return function(y) { return x * y; };
}

const double = multiplyBy(2);
```

3. 防抖节流：保存 timer 变量

4. 柯里化：函数式编程

5. 事件处理：保存状态

内存泄漏问题：

- 闭包持有外部函数作用域引用，会导致变量无法被垃圾回收

- 解决方案：手动解除引用（设置为 `null`）

1.3 原型与继承

题目：JavaScript 如何实现继承？画出原型链结构。

答案：

原型链图示：

text

实例对象 `__proto__` → 构造函数.prototype `__proto__` → `Object.prototype` `__proto__` → `null`

五种继承方式：

1. 原型链继承

javascript

```
function Parent() { this.colors = ['red']; }
function Child() {}
Child.prototype = new Parent();
const child = new Child();
// 缺点：所有实例共享引用类型属性
```

2. 构造函数继承

javascript

```
function Child() {
  Parent.call(this);
}
// 缺点：无法继承原型上的方法
```

3. 组合继承（最常用）

javascript

```
function Child() {
  Parent.call(this);
}
Child.prototype = new Parent();
Child.prototype.constructor = Child;
```

4. 寄生组合继承（最优）

javascript

```
function inherit(child, parent) {
  const prototype = Object.create(parent.prototype);
  prototype.constructor = child;
  child.prototype = prototype;
}
```

5. ES6 Class 继承

javascript

```
class Child extends Parent {
  constructor(...args) {
    super(...args);
  }
}
```

1.4 this 指向

题目：call、apply、bind 的区别？手写实现 bind。

答案：

区别对比：

方法	参数传递	执行时机	返回
call	逐个传递	立即执行	函数调用的返回值
apply	数组传递	立即执行	函数调用的返回值
bind	逐个传递	返回新函数	绑定后的函数

手写 bind:

```
javascript

Function.prototype.myBind = function(context, ...args) {
  if (typeof this !== 'function') {
    throw new TypeError('Bind must be called on a function');
  }
  const fn = this;
  return function bound(...newArgs) {
    // 判断是否通过 new 调用
    if (this instanceof bound) {
      return new fn(...args, ...newArgs);
    }
    return fn.apply(context, [...args, ...newArgs]);
  };
};
```

手写 call:

```
javascript

Function.prototype.myCall = function(context, ...args) {
  context = context || globalThis;
  const fnSymbol = Symbol();
  context[fnSymbol] = this;
  const result = context[fnSymbol](...args);
  delete context[fnSymbol];
  return result;
};
```

手写 apply:

```
javascript

Function.prototype.myApply = function(context, args) {
  context = context || globalThis;
  const fnSymbol = Symbol();
  context[fnSymbol] = this;
  const result = context[fnSymbol](...args);
  delete context[fnSymbol];
  return result;
};
```

1.5 Promise

题目：手写一个符合 Promise/A+ 规范的 Promise

答案：

javascript

```
class MyPromise {
  static PENDING = 'pending';
  static FULFILLED = 'fulfilled';
  static REJECTED = 'rejected';

  constructor(executor) {
    this.status = MyPromise.PENDING;
    this.value = null;
    this.reason = null;
    this.onFulfilledCallbacks = [];
    this.onRejectedCallbacks = [];

    const resolve = (value) => {
      if (this.status === MyPromise.PENDING) {
        this.status = MyPromise.FULFILLED;
        this.value = value;
        this.onFulfilledCallbacks.forEach(fn => fn());
      }
    };

    const reject = (reason) => {
      if (this.status === MyPromise.PENDING) {
        this.status = MyPromise.REJECTED;
        this.reason = reason;
        this.onRejectedCallbacks.forEach(fn => fn());
      }
    };

    try {
      executor(resolve, reject);
    } catch (error) {
      reject(error);
    }
  }

  then(onFulfilled, onRejected) {
    onFulfilled = typeof onFulfilled === 'function' ? onFulfilled : value => value;
    onRejected = typeof onRejected === 'function' ? onRejected : reason => { throw reason; };

    const promise2 = new MyPromise((resolve, reject) => {
      if (this.status === MyPromise.FULFILLED) {
        setTimeout(() => {
          try {
            const x = onFulfilled(this.value);
            resolvePromise(promise2, x, resolve, reject);
          } catch (error) {
            reject(error);
          }
        });
      }

      if (this.status === MyPromise.REJECTED) {
        setTimeout(() => {
          try {
            const x = onRejected(this.reason);
            resolvePromise(promise2, x, resolve, reject);
          } catch (error) {
            reject(error);
          }
        });
      }

      if (this.status === MyPromise.PENDING) {
        this.onFulfilledCallbacks.push(() => {
          setTimeout(() => {
            try {
              const x = onFulfilled(this.value);
              resolvePromise(promise2, x, resolve, reject);
            } catch (error) {
              reject(error);
            }
          });
        });
      }
    });
  }
}
```

```

        } catch (error) {
            reject(error);
        }
    });
});

this.onRejectedCallbacks.push(() => {
    setTimeout(() => {
        try {
            const x = onRejected(this.reason);
            resolvePromise(promise2, x, resolve, reject);
        } catch (error) {
            reject(error);
        }
    });
});
});
}
});

return promise2;
}

static resolve(value) {
    if (value instanceof MyPromise) return value;
    return new MyPromise(resolve => resolve(value));
}

static reject(reason) {
    return new MyPromise( (_, reject) => reject(reason));
}

static all(promises) {
    return new MyPromise((resolve, reject) => {
        const results = [];
        let completed = 0;

        promises.forEach((promise, index) => {
            MyPromise.resolve(promise).then(value => {
                results[index] = value;
                completed++;
                if (completed === promises.length) resolve(results);
            }).catch(reject);
        });
    });
}

static race(promises) {
    return new MyPromise((resolve, reject) => {
        promises.forEach(promise => {
            MyPromise.resolve(promise).then(resolve).catch(reject);
        });
    });
}

function resolvePromise(promise2, x, resolve, reject) {
    if (promise2 === x) {
        reject(new TypeError('Chaining cycle detected'));
    }

    if (x instanceof MyPromise) {
        x.then(resolve, reject);
    } else {
        resolve(x);
    }
}
}

```

1.6 深浅拷贝

题目：手写一个支持循环引用的深拷贝函数

答案：

```
javascript

function deepClone(value, hash = new WeakMap()) {
  // 处理基本类型和 null
  if (value === null || typeof value !== 'object') {
    return value;
  }

  // 处理循环引用
  if (hash.has(value)) {
    return hash.get(value);
  }

  // 处理日期
  if (value instanceof Date) {
    return new Date(value);
  }

  // 处理正则
  if (value instanceof RegExp) {
    return new RegExp(value);
  }

  // 处理数组和对象
  const clone = Array.isArray(value) ? [] : {};
  hash.set(value, clone);

  // 使用 Reflect.ownKeys 处理 Symbol 和不可枚举属性
  Reflect.ownKeys(value).forEach(key => {
    clone[key] = deepClone(value[key], hash);
  });

  return clone;
}
```

2. React 原理与生态

2.1 setState 同步还是异步？

答案：

结论：取决于调用环境

场景	行为	原因
React 生命周期中	异步	批量更新优化性能
React 合成事件中	异步	同上
setTimeout 中	同步	脱离 React 上下文
原生 DOM 事件中	同步	同上
Promise.then 中	同步	同上

示例验证：

```
javascript

class Demo extends React.Component {
  state = { count: 0 };
}
```

```

handleClick = () => {
  this.setState({ count: this.state.count + 1 });
  console.log(this.state.count); // 0 (异步)

  setTimeout(() => {
    this.setState({ count: this.state.count + 1 });
    console.log(this.state.count); // 2 (同步)
  }, 0);
};

render() {
  return <button onClick={this.handleClick}>Click</button>;
}
}

```

源码层面：

- React 通过 `isBatchingUpdates` 标志位判断是否批量更新
- 合成事件和生命周期中，`isBatchingUpdates = true`
- 异步回调中，`isBatchingUpdates = false`

2.2 React Fiber 架构

题目：React Fiber 架构解决了什么问题？原理是什么？

答案：

解决的核心问题：

React 15 的递归更新无法中断，当组件树很大时会阻塞主线程，导致页面掉帧、用户交互卡顿。

Fiber 核心设计：

1. **可中断的更新：**将同步的递归渲染拆分成多个小任务单元
2. **任务优先级：**高优先级任务（用户输入）可打断低优先级任务（数据更新）
3. **双缓存技术：**内存中构建新 Fiber 树，完成后一次性替换

工作流程：

```

text

Render 阶段（可中断）      Commit 阶段（不可中断）
    ↓                        ↓
beginWork → completeWork → 渲染 DOM → 执行副作用
   (构建 Fiber 树)          (一次性提交)

```

Fiber 节点结构：

```

javascript

{
  tag: WorkTag,           // 组件类型
  key: null,              // 唯一标识
  elementType: null,     // 元素类型
  type: null,             // 函数/类组件本身
  stateNode: null,        // 真实 DOM 节点
  return: null,           // 父节点
  child: null,            // 第一个子节点
  sibling: null,           // 兄弟节点
  alternate: null         // 上一次更新的 Fiber
}

```

与 Vue 对比：

- Vue 基于响应式依赖追踪，可以更细粒度更新
- React 需要从根节点开始协调，Fiber 架构是必要优化

2.3 Hooks 原理

题目：为什么 Hooks 不能在条件语句中调用？useState 是如何工作的？

答案：

核心原因：Hooks 依赖调用顺序

React 内部用一个链表（或数组）存储 Hooks 状态：

```
javascript

// React 内部简化实现
let workInProgressHook = null;
let isMount = true;

function useState(initialState) {
  let hook;
  if (isMount) {
    hook = {
      memoizedState: initialState,
      next: null,
      queue: []
    };
    // 挂载到链表中
  } else {
    hook = workInProgressHook;
    workInProgressHook = workInProgressHook.next;
  }
  return [hook.memoizedState, dispatchAction];
}
```

错误示例：

```
javascript

function Counter() {
  const [count, setCount] = useState(0);
  if (count > 0) {
    const [other, setOther] = useState(0); // ❌ 条件性调用
  }
  // count 变化时, Hooks 顺序错乱, 状态对应出错
}
```

正确做法：

- 将所有 Hooks 放在组件顶层
- 使用 useMemo / useCallback 来条件性执行逻辑

2.4 useEffect vs useLayoutEffect

对比项	useEffect	useLayoutEffect
执行时机	浏览器绘制后	DOM 更新后、绘制前
阻塞渲染	不阻塞	阻塞
使用场景	数据获取、订阅、日志	测量 DOM、无闪烁更新
推荐程度	优先使用	仅在需要时使用

源码层面：

- `useEffect` 在 `commit` 阶段异步执行，放入调度队列
- `useLayoutEffect` 在 `commit` 阶段同步执行，会阻塞浏览器绘制

2.5 React.memo vs useMemo vs useCallback

API	作用	返回	使用场景
<code>React.memo</code>	组件缓存	新组件	防止子组件不必
<code>useMemo</code>	值缓存	计算结果	昂贵的计算
<code>useCallback</code>	函数缓存	函数本身	传递给子组件的

示例：

```
javascript

// 子组件优化
const Child = React.memo(({ value, onChange }) => {
  // 只有当 value 或 onChange 变化时才重渲染
});

function Parent() {
  const [count, setCount] = useState(0);

  // useCallback 保证函数引用不变
  const handleChange = useCallback(() => {
    console.log('changed');
  }, []); // 依赖为空, 永远不变

  // useMemo 缓存计算结果
  const expensiveValue = useMemo(() => {
    return computeExpensiveValue(count);
  }, [count]);

  return <Child value={expensiveValue} onChange={handleChange} />;
}
```

2.6 Redux 原理

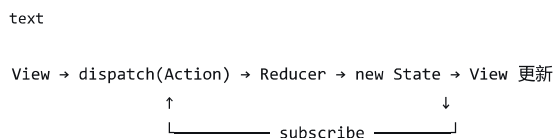
题目：Redux 的工作原理？手写 `createStore` 和 `applyMiddleware`。

答案：

核心原则：

1. **单一数据源**：整个应用状态存在一个 Store
2. **State 只读**：只能通过触发 Action 来修改
3. **纯函数修改**：Reducer 必须是纯函数

流程图：



手写 `createStore`：

javascript

```
function createStore(reducer, preloadedState, enhancer) {
  if (enhancer) {
    return enhancer(createStore)(reducer, preloadedState);
  }

  let state = preloadedState;
  let listeners = [];

  function getState() {
    return state;
  }

  function dispatch(action) {
    state = reducer(state, action);
    listeners.forEach(listener => listener());
    return action;
  }

  function subscribe(listener) {
    listeners.push(listener);
    return () => {
      listeners = listeners.filter(l => l !== listener);
    };
  }

  // 初始化
  dispatch({ type: '@@redux/INIT' });

  return { getState, dispatch, subscribe };
}
```

手写 applyMiddleware:

javascript

```
function applyMiddleware(...middlewares) {
  return (createStore) => (reducer, preloadedState) => {
    const store = createStore(reducer, preloadedState);
    let dispatch = store.dispatch;

    const middlewareAPI = {
      getState: store.getState,
      dispatch: (action) => dispatch(action)
    };

    const chain = middlewares.map(middleware => middleware(middlewareAPI));
    dispatch = compose(...chain)(store.dispatch);

    return { ...store, dispatch };
  };
}

// 中间件示例: thunk
const thunk = ({ dispatch, getState }) => (next) => (action) => {
  if (typeof action === 'function') {
    return action(dispatch, getState);
  }
  return next(action);
};
```

3. 算法与手写代码

3.1 反转链表 (LeetCode 206)

javascript

```
// 迭代解法
function reverseList(head) {
  let prev = null;
  let curr = head;

  while (curr) {
    const next = curr.next;
    curr.next = prev;
    prev = curr;
    curr = next;
  }

  return prev;
}

// 递归解法
function reverseList(head) {
  if (!head || !head.next) return head;
  const newHead = reverseList(head.next);
  head.next.next = head;
  head.next = null;
  return newHead;
}
```

3.2 LRU 缓存 (LeetCode 146)

javascript

```
class LRUCache {
  constructor(capacity) {
    this.capacity = capacity;
    this.cache = new Map(); // 使用 Map 保证插入顺序
  }

  get(key) {
    if (!this.cache.has(key)) return -1;
    const value = this.cache.get(key);
    // 刷新到最新
    this.cache.delete(key);
    this.cache.set(key, value);
    return value;
  }

  put(key, value) {
    if (this.cache.has(key)) {
      this.cache.delete(key);
    }
    this.cache.set(key, value);

    if (this.cache.size > this.capacity) {
      // 删除最久未使用的 (Map 中第一个)
      const firstKey = this.cache.keys().next().value;
      this.cache.delete(firstKey);
    }
  }
}
```

3.3 防抖与节流

javascript

```
// 防抖: 延迟执行, 连续触发只执行最后一次
function debounce(fn, delay, immediate = false) {
```

```

let timer = null;

return function(...args) {
  if (timer) clearTimeout(timer);

  if (immediate && !timer) {
    fn.apply(this, args);
  }

  timer = setTimeout(() => {
    fn.apply(this, args);
    timer = null;
  }, delay);
};
}

// 节流: 固定时间间隔执行一次
function throttle(fn, delay, trailing = true) {
  let timer = null;
  let lastCall = 0;

  return function(...args) {
    const now = Date.now();

    if (now - lastCall >= delay) {
      fn.apply(this, args);
      lastCall = now;
    } else if (trailing && !timer) {
      timer = setTimeout(() => {
        fn.apply(this, args);
        lastCall = Date.now();
        timer = null;
      }, delay - (now - lastCall));
    }
  };
}

```

3.4 发布订阅模式 (Event Bus)

javascript

```

class EventBus {
  constructor() {
    this.events = new Map();
  }

  on(event, callback) {
    if (!this.events.has(event)) {
      this.events.set(event, []);
    }
    this.events.get(event).push(callback);
  }

  off(event, callback) {
    if (!this.events.has(event)) return;
    if (!callback) {
      this.events.delete(event);
      return;
    }
    const callbacks = this.events.get(event);
    const index = callbacks.indexOf(callback);
    if (index !== -1) callbacks.splice(index, 1);
  }

  emit(event, ...args) {
    if (!this.events.has(event)) return;
    this.events.get(event).forEach(cb => cb(...args));
  }
}

```

```
once(event, callback) {
  const wrapper = (...args) => {
    callback(...args);
    this.off(event, wrapper);
  };
  this.on(event, wrapper);
}
```

4. 工程化 & 网络 & 性能

4.1 从 URL 到页面渲染的完整流程

text

1. DNS 解析: 域名 → IP 地址
2. TCP 连接: 三次握手
3. TLS 协商 (HTTPS) : 加密通信建立
4. 发送 HTTP 请求
5. 服务器处理并返回响应
6. 浏览器解析响应
 - ├─ 构建 DOM 树 (HTML → DOM)
 - ├─ 构建 CSSOM 树 (CSS → CSSOM)
 - ├─ 合并为渲染树 (Render Tree)
 - ├─ 布局 (Layout/Reflow) : 计算位置大小
 - └─ 绘制 (Paint) : 填充像素
7. 合成与显示

4.2 HTTP 缓存策略

javascript

```
// 强缓存: 直接从缓存读取, 不请求服务器
// 响应头: Cache-Control (优先级高于 Expires)
Cache-Control: max-age=3600 // 缓存 1 小时
Cache-Control: no-cache     // 每次都验证
Cache-Control: no-store     // 完全不缓存

// 协商缓存: 需要向服务器验证
// 响应头: ETag + Last-Modified
// 请求头: If-None-Match + If-Modified-Since

// Service Worker 精细化缓存
self.addEventListener('fetch', (event) => {
  event.respondWith(
    caches.match(event.request)
      .then(response => response || fetch(event.request))
  );
});
```

4.3 首屏性能优化方案

优化方向	具体措施	预期收益
网络层	CDN 加速、HTTP/2、预连接	减少 RTT
资源层	代码分割、Tree Shaking、图片懒加载	减少首屏资源
渲染层	SSR/SSG、关键 CSS 内联、骨架屏	减少白屏时间
缓存层	Service Worker、强缓存策略	二次访问秒开

4.4 XSS 与 CSRF 防范

XSS (跨站脚本攻击) :

- 类型: 反射型、存储型、DOM 型
- 防范:
 - 输入过滤 + 输出转义 (`textContent` 替代 `innerHTML`)
 - CSP (内容安全策略)
 - Cookie 设置 `HttpOnly` 和 `SameSite`

CSRF (跨站请求伪造) :

- 原理: 利用用户登录状态发起恶意请求
- 防范:
 - CSRF Token (双重 Cookie 验证)
 - `SameSite` Cookie 属性
 - 验证 `Referer/Origin`

本文档涵盖大厂前端面试 80% 以上核心考点, 建议反复练习手写代码部分。

本回答由 AI 生成, 内容仅供参考, 请仔细甄别。